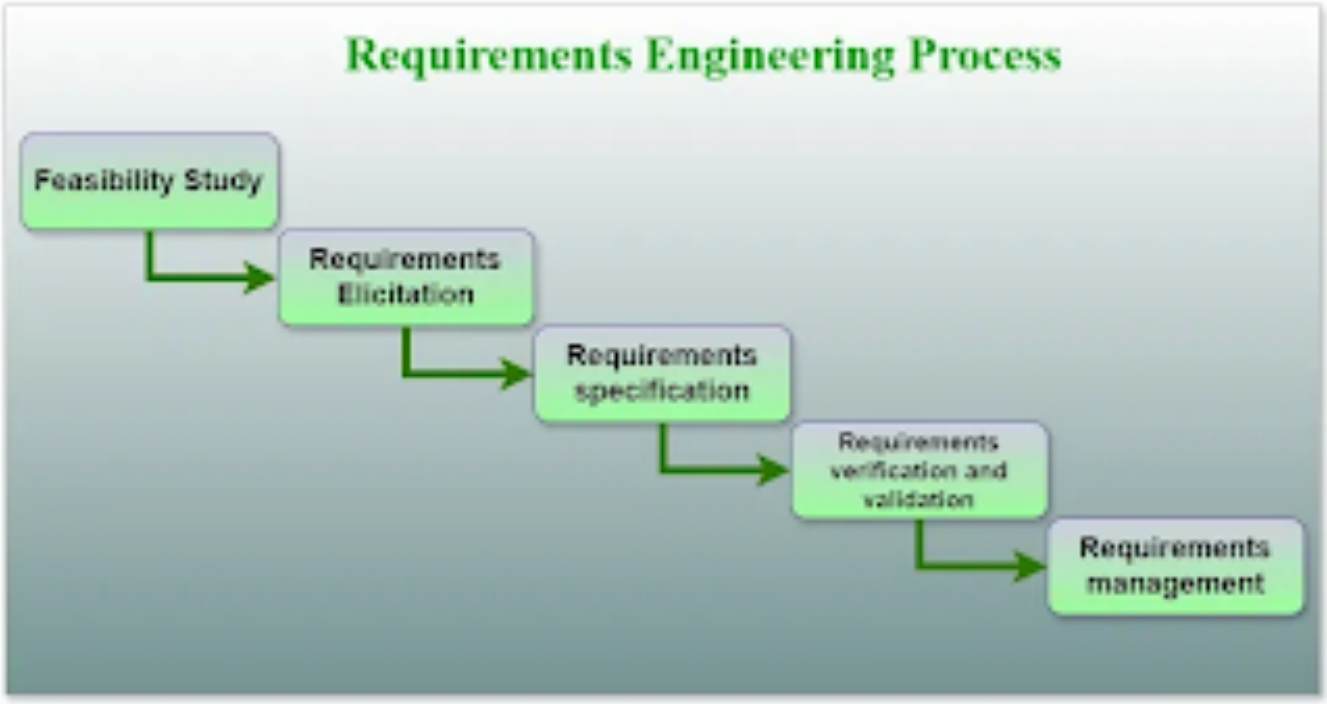
CHAPTER 2

Software Requirements Analysis and Modeling

Requirement Engineering, Requirement Modeling,
Scenario based model. (Usecase & Activity diagram)
Software Requirement Specification document format(IEEE), Data flow diagram

Requirements Engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.



1.. Feasibility Study

The feasibility study mainly concentrates on below five mentioned areas below. Among these Economic Feasibility Study is the most important part of the feasibility analysis and the Legal Feasibility Study is less considered feasibility analysis.

1. Technical Feasibility: In Technical Feasibility current resources both hardware software along required technology are analyzed/assessed to develop the project. This technical feasibility study reports whether there are correct required resources and technologies that will be used for project development.

2. Operational Feasibility: In Operational Feasibility degree of providing service to requirements is analyzed along with how easy the product will be to operate and maintain after deployment.

3. Economic Feasibility: In the Economic Feasibility study cost and benefit of the project are analyzed.

4. Legal Feasibility: In legal feasibility, the project is ensured to comply with all relevant laws, regulations, and standards. It identifies any legal constraints that could impact the project and reviews existing contracts and agreements to assess their effect on the project's execution.

5. Schedule Feasibility: In schedule feasibility, the project timeline is evaluated to determine if it is realistic and achievable. Significant milestones are identified, and deadlines are established to track progress effectively. Resource availability is assessed to ensure that the necessary resources are accessible to meet the project schedule. Furthermore, any time constraints that might affect project delivery are considered to ensure timely completion. This focus on schedule feasibility is crucial for the successful planning and execution of a project.

2. Requirements Elicitation

It is related to the various ways used to gain knowledge about the project domain and requirements. The various sources of domain knowledge include customers, business manuals, the existing software of the same type, standards, and other stakeholders of the project. The techniques used for requirements elicitation include interviews, brainstorming, task analysis, Delphi technique, prototyping, etc. Requirements elicitation is the process of gathering information about the needs and expectations of stakeholders for a software system.

3. Requirements Specification

This activity is used to produce formal software requirement models. All the requirements including the functional as well as the non-functional requirements and the constraints are specified by these models in totality.

Requirements specification is the process of documenting the requirements identified in the analysis step in a clear, consistent, and unambiguous manner. This step also involves prioritizing and grouping the requirements into manageable chunks.

The goal of this step is to create a clear and comprehensive document that describes the requirements for the software system. This document should be understandable by both the development team and the stakeholders.

Several types of requirements are commonly specified in this step, including

1. Functional Requirements: These describe what the software system should do. They specify the functionality that the system must provide, such as input validation, data storage, and user interface.

2. Non-Functional Requirements: These describe how well the software system should do it. They specify the quality attributes of the system, such as performance, reliability, usability, and security.

3. Constraints: These describe any limitations or restrictions that must be considered when developing the software system.

4. Acceptance Criteria: These describe the conditions that must be met for the software system to be considered complete and ready for release.

To make the requirements specification clear, the requirements should be written in a natural language and use simple terms, avoiding technical jargon, and using a consistent format throughout the document. It is also important to use diagrams, models, and other visual aids to help communicate the requirements effectively.

Once the requirements are specified, they must be reviewed and validated by the stakeholders and development team to ensure that they are complete, consistent, and accurate.

4. Requirements Verification and Validation

Verification: It refers to the set of tasks that ensures that the software correctly implements a specific function.

Validation: It refers to a different set of tasks that ensures that the software that has been built is traceable to customer requirements. If requirements are not validated, errors in the requirement definitions would propagate to the successive stages resulting in a lot of modification and rework.

Requirements verification and validation (V&V) is the process of checking that the requirements for a software system are complete, consistent, and accurate and that they meet the needs and expectations of the stakeholders.

5. Requirements Management

Requirement management is the process of analyzing, documenting, tracking, prioritizing, and agreeing on the requirement and controlling the communication with relevant stakeholders. It should be ensured that the SRS is as modifiable as possible to incorporate changes in requirements specified by the end users at later stages too.

Requirements management is the process of managing the requirements throughout the software development life cycle, including tracking and controlling changes, and ensuring that the requirements are still valid and relevant.

Advantages of Requirements Engineering Process

- Helps ensure that the software being developed meets the needs and expectations of the stakeholders
- Can help identify potential issues or problems early in the development process, allowing for adjustments to be made before significant
- Helps ensure that the software is developed in a cost-effective and efficient manner
- Can improve communication and collaboration between the development team and stakeholders
- Helps to ensure that the software system meets the needs of all stakeholders.
- Provides an unambiguous description of the requirements, which helps to reduce misunderstandings and errors.
- Helps to identify potential conflicts and contradictions in the requirements, which can be resolved before the software development process begins.

- Helps to ensure that the software system is delivered on time, within budget, and to the required quality standards.
- Provides a solid foundation for the development process, which helps to reduce the risk of failure.

Disadvantages of Requirements Engineering Process

- Can be time-consuming and costly, particularly if the requirements-gathering process is not well-managed
- Can be difficult to ensure that all stakeholders' needs and expectations are taken into account
- It Can be challenging to ensure that the requirements are clear, consistent, and complete
- Changes in requirements can lead to delays and increased costs in the development process.
- As a best practice, Requirements engineering should be flexible, adaptable, and should be aligned with the overall project goals.
- It can be time-consuming and expensive, especially if the requirements are complex.
- It can be difficult to elicit requirements from stakeholders who have different needs and priorities.
- Requirements may change over time, which can result in delays and additional costs.
- There may be conflicts between stakeholders, which can be difficult to resolve.
- It may be challenging to ensure that all stakeholders understand and agree on the requirements.

Stages in Software Engineering Process

Requirements engineering is a critical process in software engineering that involves identifying, analyzing, documenting, and managing the requirements of a software system. The requirements engineering process consists of the following stages:

- Elicitation: In this stage, the requirements are gathered from various stakeholders such as customers, users, and domain experts. The aim is to identify the features and functionalities that the software system should provide.
- Analysis: In this stage, the requirements are analyzed to determine their feasibility, consistency, and completeness. The aim is to identify any conflicts or contradictions in the requirements and resolve them.
- Specification: In this stage, the requirements are documented in a clear, concise, and unambiguous manner. The aim is to provide a detailed description of the requirements that can be understood by all stakeholders.
- Validation: In this stage, the requirements are reviewed and validated to ensure that they meet the needs of all stakeholders. The aim is to ensure that the requirements are

accurate, complete, and consistent.

- Management: In this stage, the requirements are managed throughout the software development lifecycle. The aim is to ensure that any changes or updates to the requirements are properly documented and communicated to all stakeholders.
- Effective requirements engineering is crucial to the success of software development projects. It helps ensure that the software system meets the needs of all stakeholders and is delivered on time, within budget, and to the required quality standards.

Requirements modelling

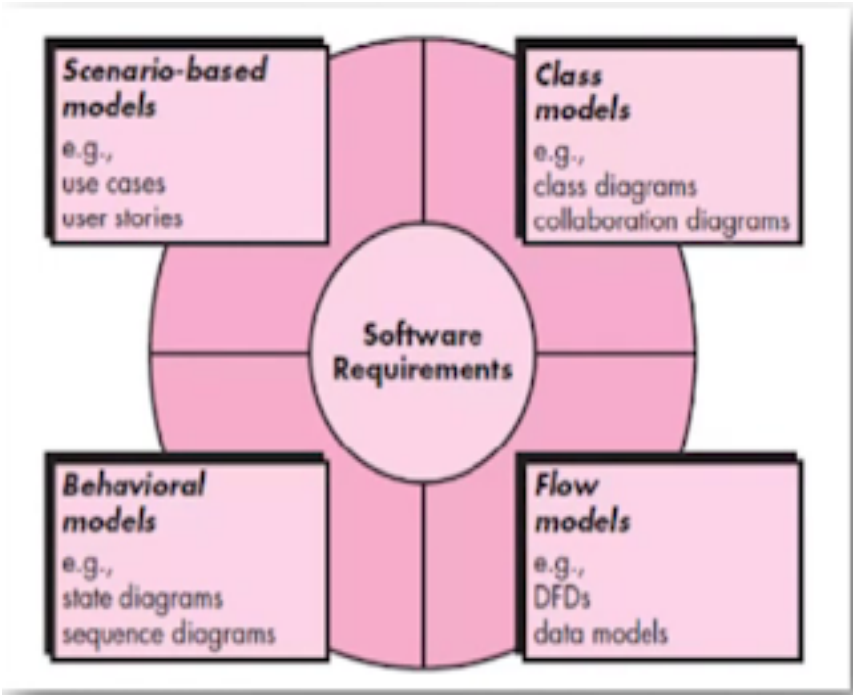
Requirements modeling refers to the process of creating a visual representation of system requirements. It helps stakeholders understand the system's functionality and ensures alignment between the business and technical teams. Requirements modeling software enables the capture, analysis, and management of requirements, facilitating better collaboration and communication throughout the project lifecycle.

Requirement modelling is important because:

1. Clarifies System Expectations: Requirements modeling provides a clear visualization of system behavior, user interactions, and data flow, reducing ambiguity and miscommunication.

2. Improves Traceability: With the right requirements modeling solutions, you can track the evolution of requirements, ensuring they are accurately implemented in the final product.

3. Enhances Collaboration: By using visual models and requirements modeling systems, teams can effectively collaborate on defining and refining requirements.

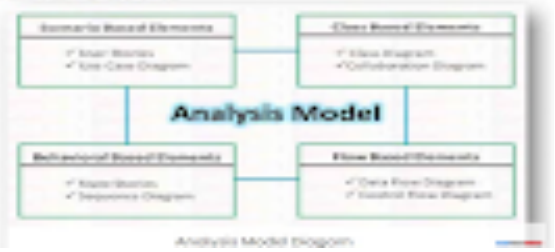


Requirement Analysis

- Requirement analysis is significant and essential activity after elicitation.
- This activity reviews all requirements and may provide a graphical view of the entire system.
- After the completion of the analysis, the understandability of the project may improve significantly.

➢**Requirement Analysis Model / Elements of Requirement Model**

1. Scenario based Modelling
2. Class based Modelling
3. Flow oriented Modelling
4. Behavior Modelling



Scenario Based Modelling / Element

➢For building the design & analysis model it is important for software engineer to understan how end users & other actors want to interact with the system.

➢Here, creation of scenario by using Use case Diagram, Activity Diagram & User Stories.

1. Use case Diagram:

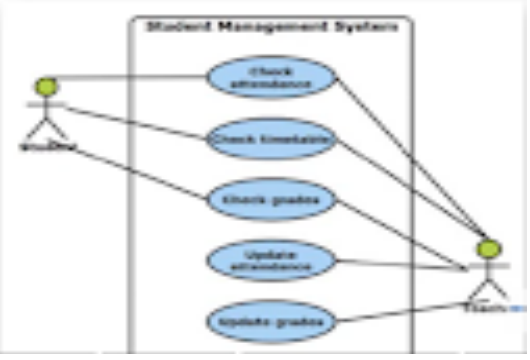
- A use case diagram is used to represent the dynamic behavior of a system.

: Actors

: Use cases

: Communication Link

: System Boundary

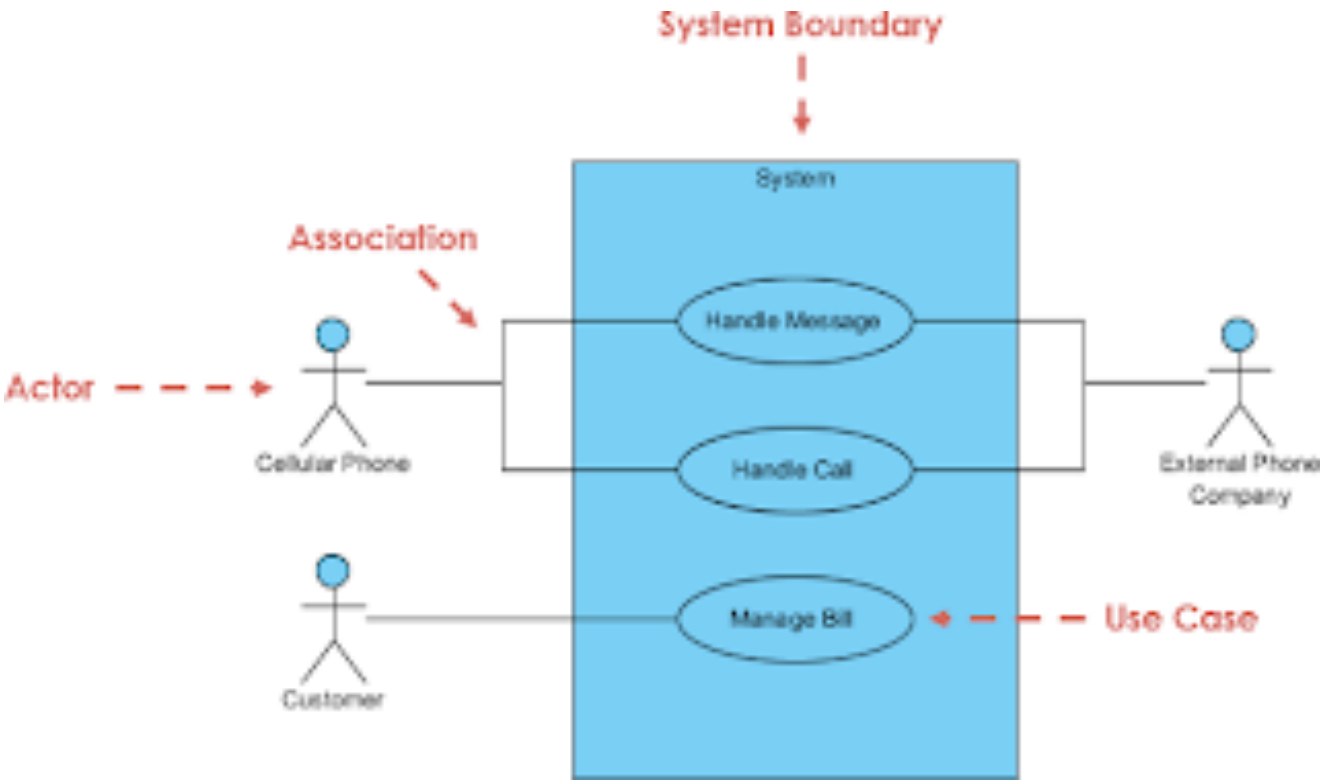


Use case diagrams are typically developed in the early stage of development and people often apply use case modeling for the following purposes:

- Specify the context of a system
- Capture the requirements of a system
- Validate a systems architecture
- Drive implementation and generate test cases
- Developed by analysts together with domain experts

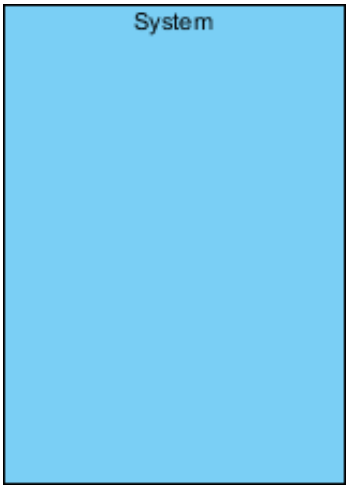
Use Case Diagram at a Glance

A standard form of use case diagram is defined in the Unified Modeling Language as shown in the Use Case Diagram example below:



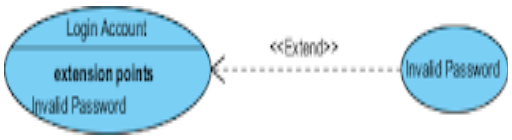
Notation Description	Visual Representation
Actor • Someone interacts with use case (system function). • Named by noun. • Actor plays a role in the business	

● Similar to the concept of user, but a user can play different roles ● For example: ● A prof. can be instructor and also researcher ● plays 2 roles with two systems ● Actor triggers use case(s). ● Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs).	
Use Case ● System function (process - automated or manual) ● Named by verb + Noun (or Noun Phrase). ● i.e. Do something ● Each Actor must be linked to a use case, while some use cases may not be linked to actors.	
Communication Link ● The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link. ● Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.	
Boundary of system	

● The system boundary is potentially the entire system as defined in the requirements document. ● For large and complex systems, each module may be the system boundary. ● For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc. ● can form a system boundary for use cases specific to each of these business functions. ● The entire system can span all of these modules depicting the overall system boundary	

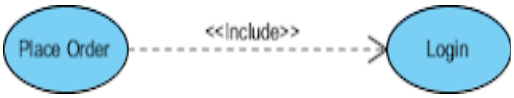
Structuring Use Case Diagram with Relationships

Use cases share different kinds of relationships. Defining the relationship between two use cases is the decision of the software analysts of the use case diagram. A relationship between two use cases is basically modeling the dependency between the two use cases. The reuse of an existing use case by using different types of relationships reduces the overall effort required in developing a system. Use case relationships are listed as the following:

Use Case Relationship	Visual Representation
Extends ● Indicates that an "Invalid Password" use case may include (subject to specified in the extension) the behavior specified by base use case "Login Account". ● Depict with a directed arrow having a dotted line. The tip of arrowhead points to the base use case and the child use case is connected at the base of the arrow.	

● The stereotype "<<extends>>" identifies as an extend relationship

Include



● When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.

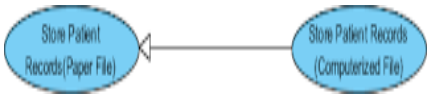
● A use case includes the functionality described in another use case as a part of its business process flow.

● A uses relationship from base use case to child use case indicates that an instance of the base use case will include the behavior as specified in the child use case.

● An include relationship is depicted with a directed arrow having a dotted line. The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow.

● The stereotype "<<include>>" identifies

Generalization



● A generalization relationship is a parent-child relationship between use cases.

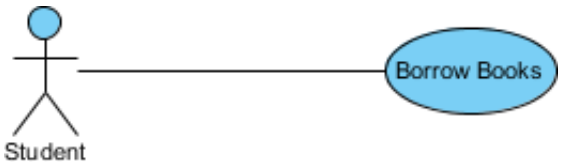
● The child use case is an enhancement of the parent use case.

- Generalization is shown as a directed arrow with a triangle arrowhead.
- The child use case is connected at the base of the arrow. The tip of the arrow is connected to the parent use case.

Use Case Examples

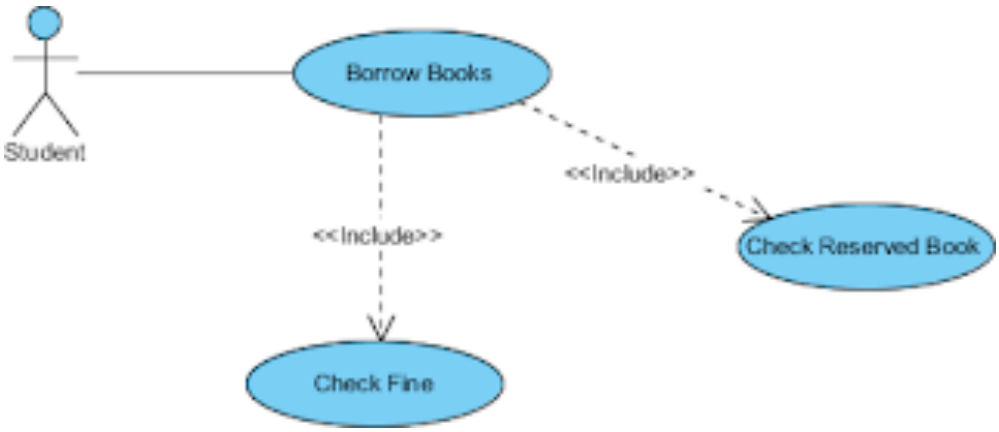
Use Case Example - Association Link

A Use Case diagram illustrates a set of use cases for a system, i.e. the actors and the relationships between the actors and use cases.



Use Case Example - Include Relationship

The include relationship adds additional functionality not specified in the base use case. The <<Include>> relationship is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



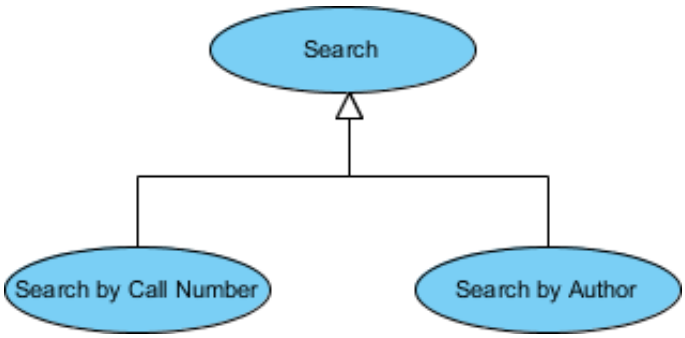
Use Case Example - Extend Relationship

The extend relationships are important because they show optional functionality or system behavior. The <<extend>> relationship is used to include optional behavior from an extending use case in an extended use case. Take a look at the use case diagram example below. It shows an extend connector and an extension point "Search".



Use Case Example - Generalization Relationship

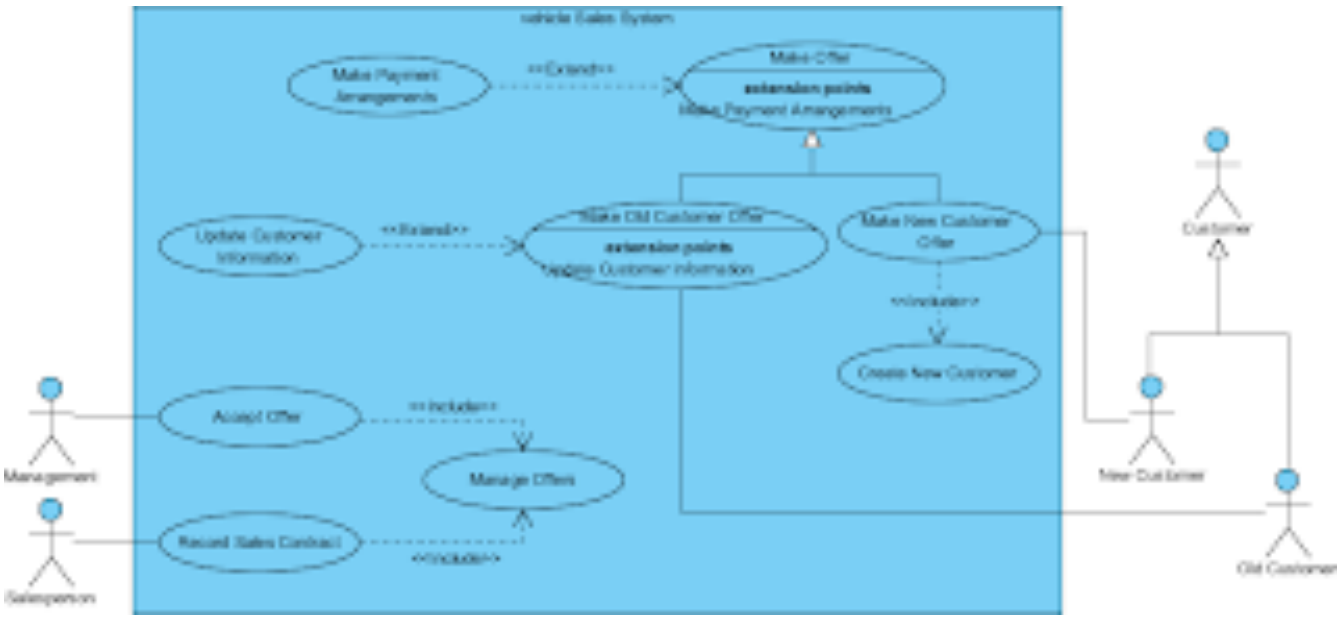
A generalization relationship means that a child use case inherits the behavior and meaning of the parent use case. The child may add or override the behavior of the parent. The figure below provides a use case example by showing two generalization connectors that connect between the three use cases.



Use Case Diagram - Vehicle Sales Systems

The figure below shows a use case diagram example for a vehicle system. As you can see even a system as big as a vehicle sales system contains not more than 10 use cases! That's the beauty of use case modeling.

The use case model also shows the use of extend and include. Besides, there are associations that connect between actors and use cases.



How to Identify Actor

Often, people find it easiest to start the requirements elicitation process by identifying the actors. The following questions can help you identify the actors of your system (Schneider and Winters - 1998):

- Who uses the system?
- Who installs the system?
- Who starts up the system?
- Who maintains the system?
- Who shuts down the system?
- What other systems use this system?
- Who gets information from this system?
- Who provides information to the system?
- Does anything happen automatically at a present time?

How to Identify Use Cases?

Identifying the Use Cases, and then the scenario-based elicitation process carries on by asking what externally visible, observable value that each actor desires. The following questions can be asked to identify use cases, once your actors have been identified (Schneider and Winters - 1998):

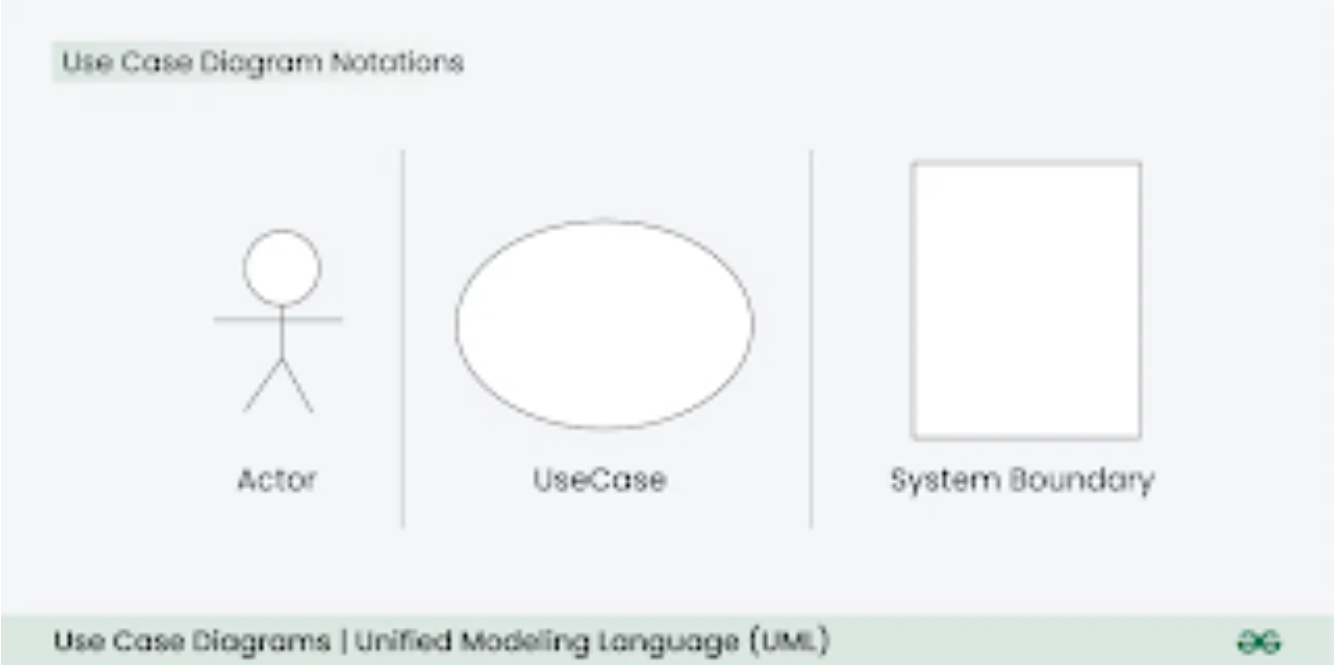
- What functions will the actor want from the system?
- Does the system store information? What actors will create, read, update or delete this information?
- Does the system need to notify an actor about changes in the internal state?
- Are there any external events the system must know about? What actor informs the system of those events?

A Use Case Diagram is a UML diagram that shows how users (actors) interact with a system and the functionalities (use cases) it provides. It gives a high-level view of the system from the user's perspective and helps define system requirements and scope.

- Represents actors (users or external systems) and the use cases they interact with.

- Shows relationships between users and system functionalities, making requirements easier to understand.

***Example**: In an Online Banking System, actors such as Customer and Admin interact with use cases like Login, Check Balance, Transfer Funds, and Manage Accounts.*



When to Apply Use Case Diagram?

Use case diagrams are used to understand, analyze, and communicate how users interact with a system.

Requirements Gathering

Used to identify and clarify user requirements and system functionalities.

- Helps understand user needs and expectations.
- Visualizes interactions between users and the system.

Communication with Stakeholders

Provides a simple representation that is easy for both technical and non-technical users to understand.

- Improves communication among stakeholders.
- Makes system functionality easier to explain.

System Design and Planning

Helps define user interactions during the design phase.

- Assists in planning system features and workflows.
- Ensures the design aligns with user requirements.

Defining System Boundaries

Clarifies what is part of the system and what is external to it.

- Identifies actors interacting with the system.
- Helps establish the scope of the system clearly.

Use Case Diagram Notations

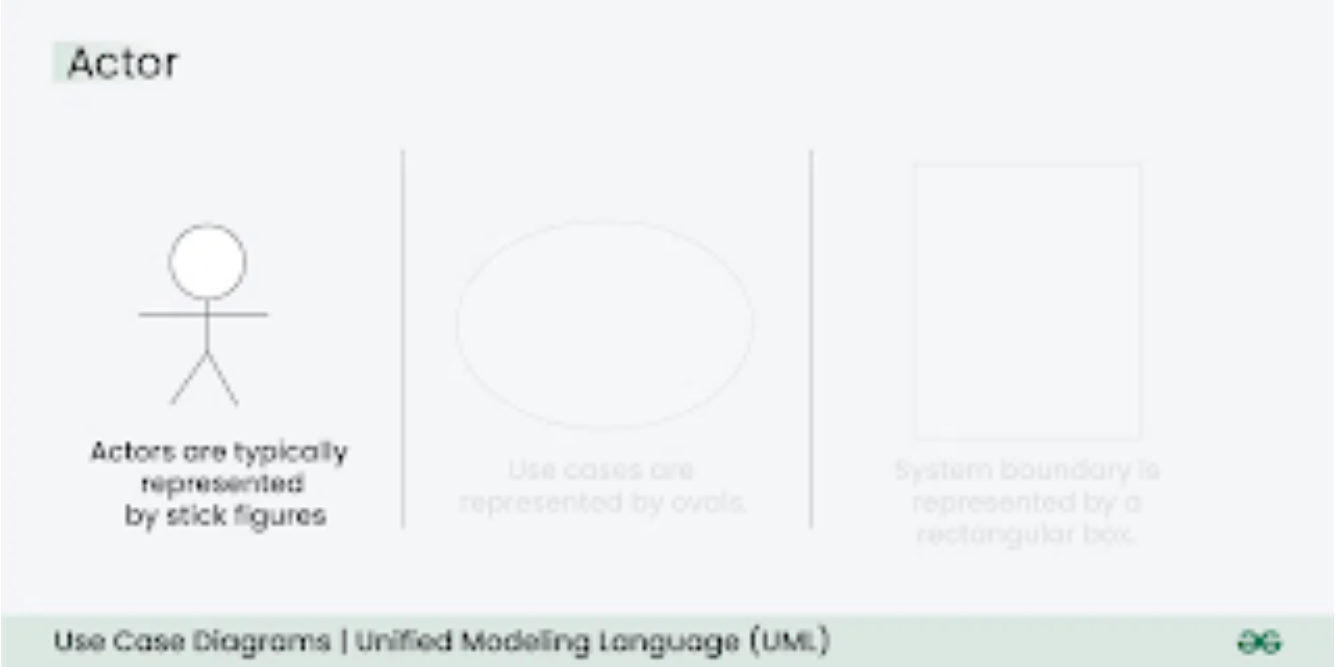
UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviors in a consistent and understandable manner.

1. Actors

Actors are external entities that interact with a system to perform specific tasks or achieve goals. They can be users, other systems, or hardware devices that initiate or participate in use cases.

- Actors interact with the system but are not part of the system itself.
- They initiate use cases and receive the results of system operations.

Example: *In an ATM System, the Customer is an actor who performs actions such as Withdraw Cash and Check Balance.*



2. Use Cases

Use Cases represent specific functionalities or services provided by a system to achieve a user's goal. They describe what the system does in response to an actor's interaction and are represented by ovals in a Use Case Diagram.

- Define the actions or functions that the system performs.
- Represent interactions between actors and system functionalities.

Example: *In an Online Shopping System, "Place Order", "Track Delivery", and "Update Product Information" are examples of use cases.*

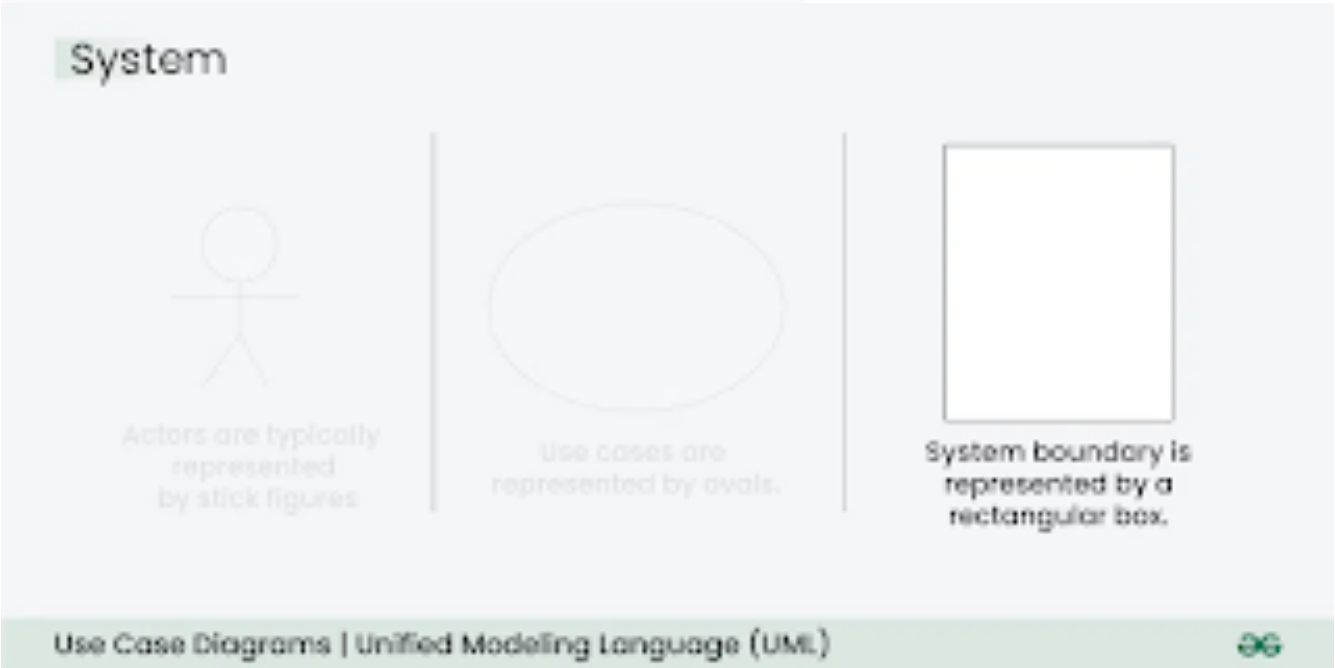


3. System Boundary

The System Boundary defines the scope and limits of the system being modeled. It separates the system's internal functionalities from external actors and entities and is usually represented by a rectangular box around the use cases.

- Identifies what is included within the system and what lies outside it.
- Helps clearly define the scope of the system.

***Example**: In an Online Shopping System, all use cases such as "Place Order" and "Track Delivery" are enclosed within the system boundary, while Customer and Admin remain outside as actors.*



Use Case Diagram Relationships

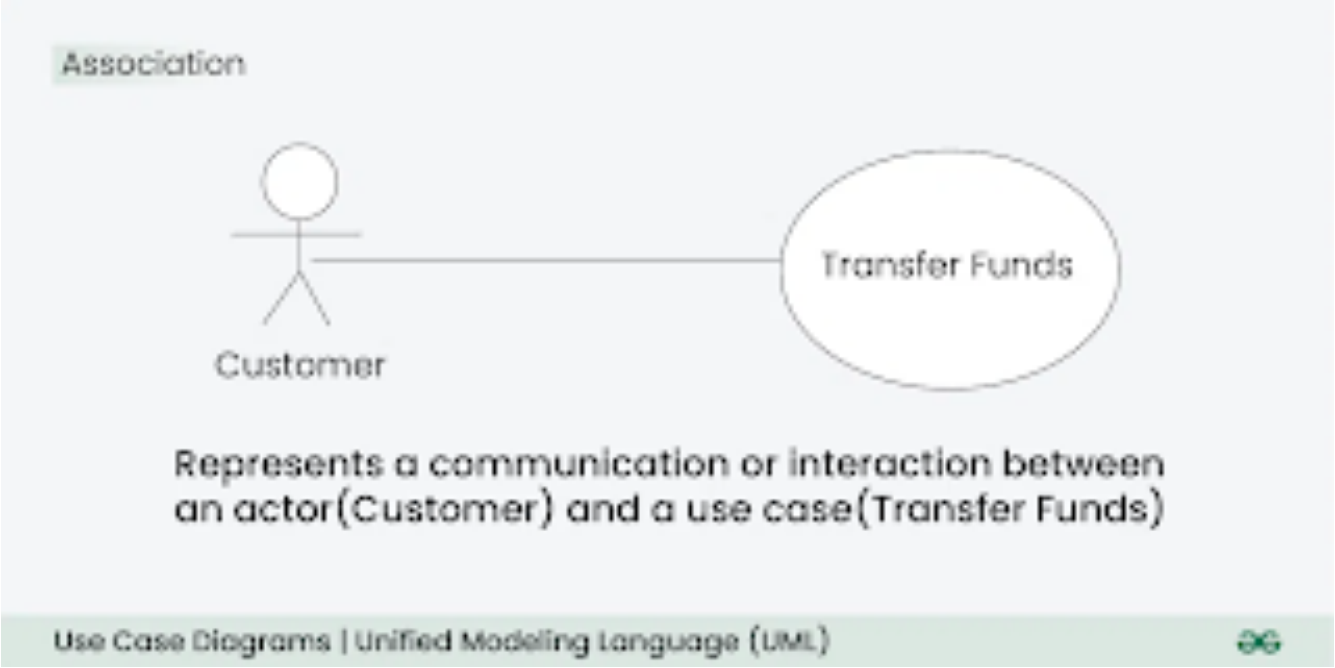
In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases.

1. Association Relationship

The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- **Actor:** Customer
- **Use Case:** Transfer Funds
- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

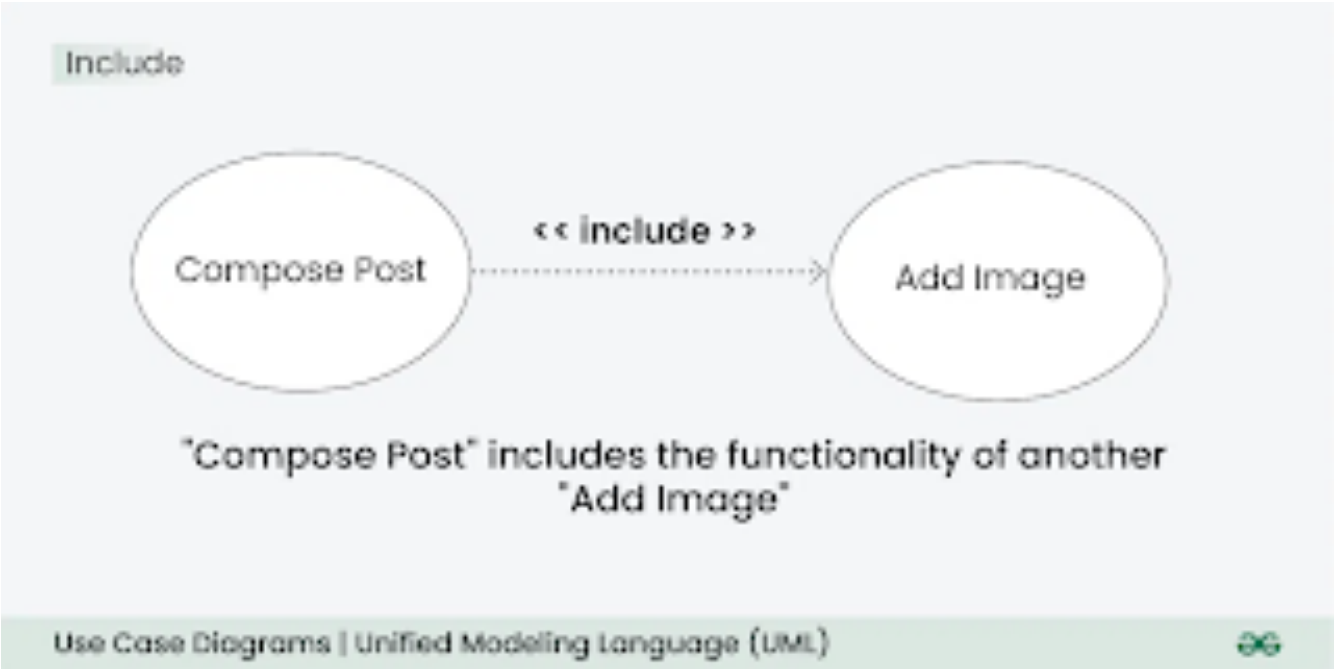


2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

Example: Social Media Posting

- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

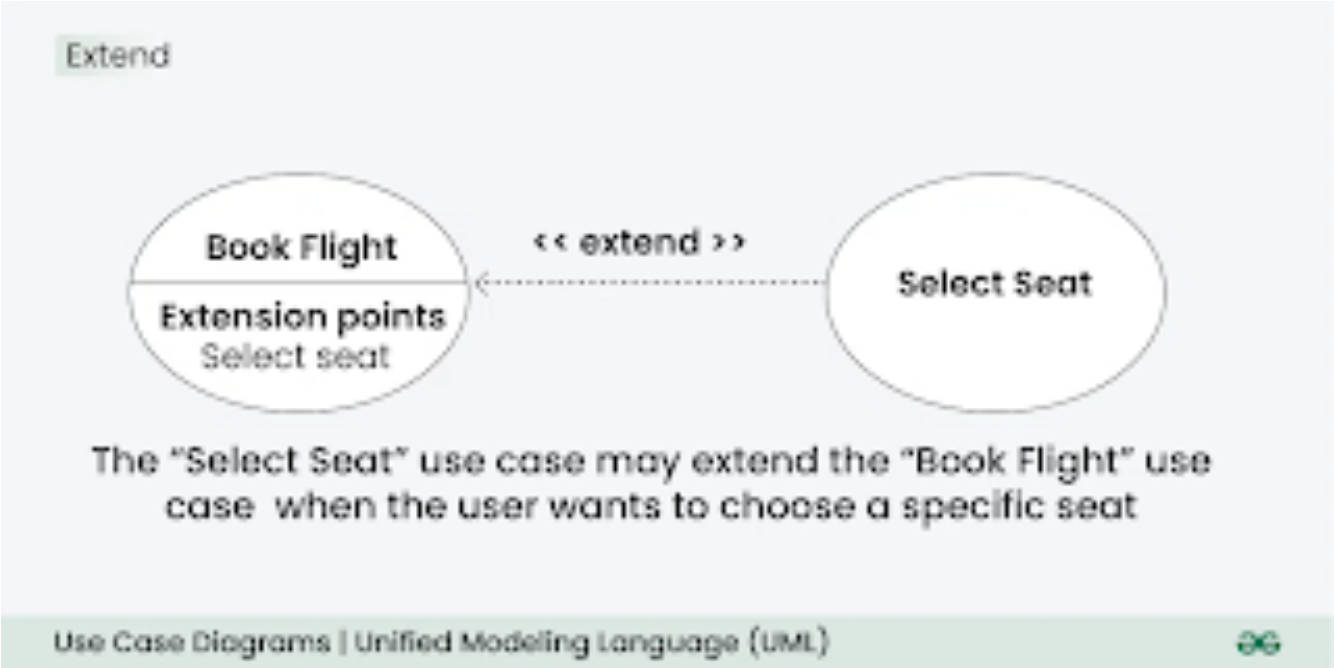


3. Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "extend." This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.



4. Generalization Relationship

The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship:** Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."

Use Case Diagram example(Online Shopping System)

Let's understand how to draw a Use Case diagram with the help of an Online Shopping System:

- Actors
- Customer

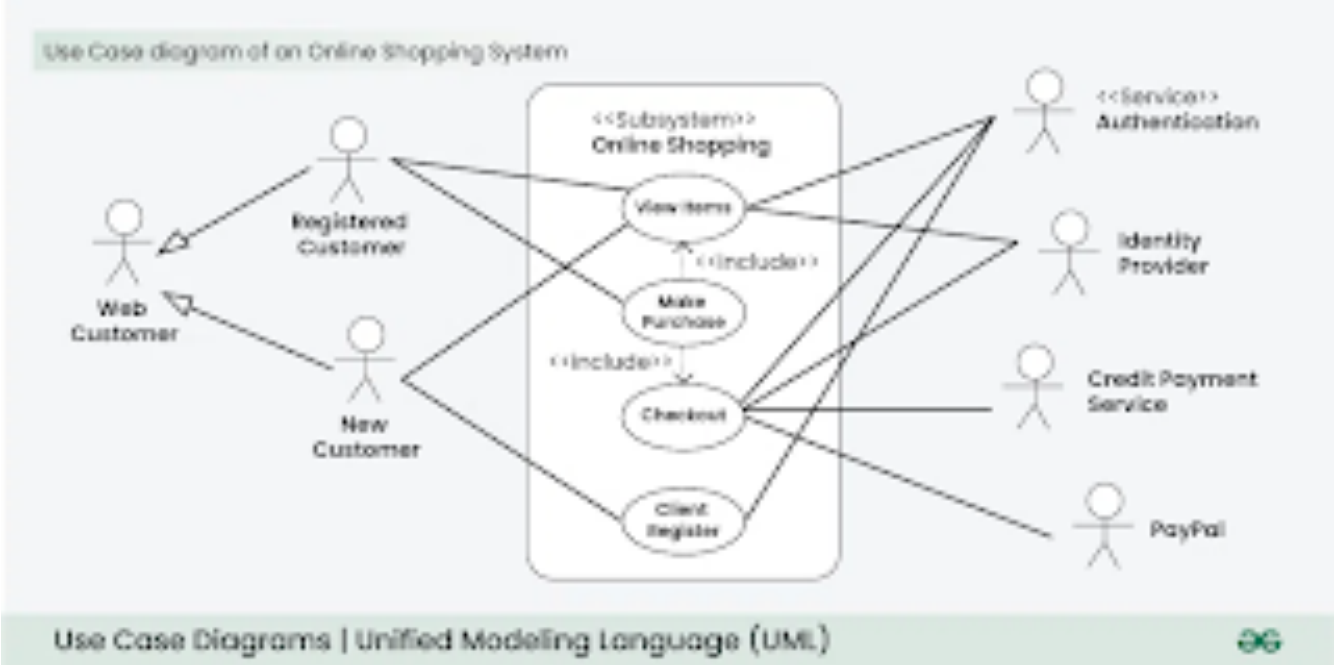
Use Cases

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

Relations

- The Customer can browse products, add items to the cart, and complete the checkout.
- The Admin can manage the inventory

The use case diagram of an Online Shopping System:



Creating a use case diagram involves five main steps: identifying system boundaries, identifying actors, identifying use cases, defining relationships, and reviewing the model.

Define the System Boundary

- Draw a large rectangle to show the scope of the system.
- Give the box a clear title at the top.
- Keep everything inside that the system must do.

Identify the Actors

- Find the people, groups, or external systems that use the system.
- Use role names instead of real personal names (e.g., "Customer" or "Cashier").
- Place these actors outside the system boundary box.
-

Identify the Use Cases

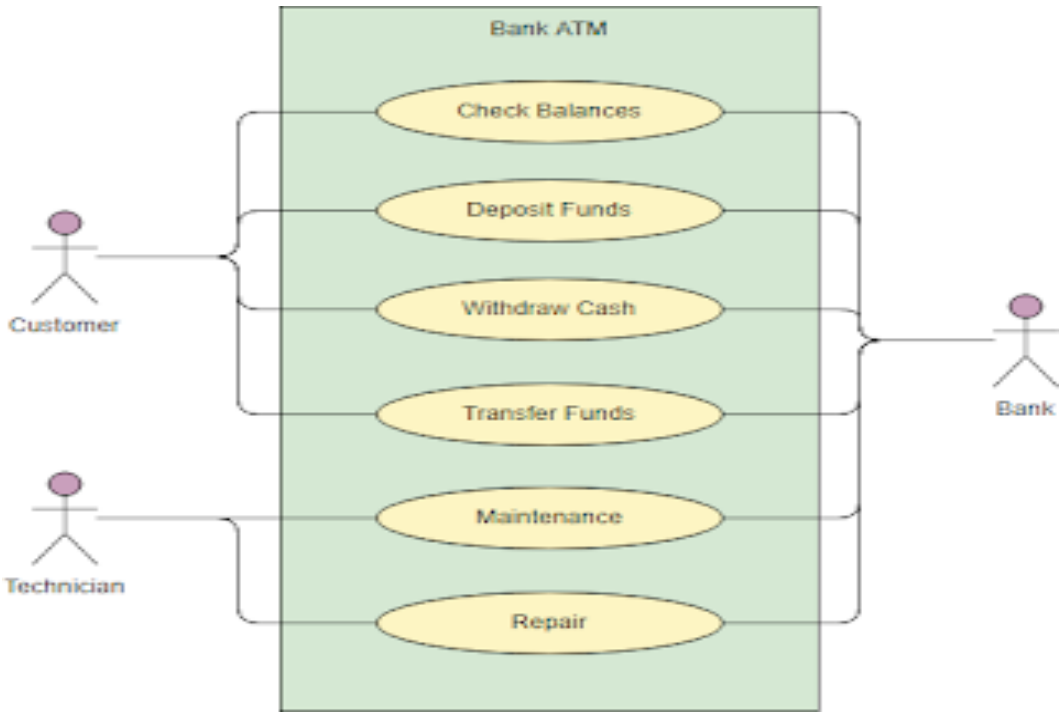
- Determine what tasks or goals each actor needs from the system.
- Name each use case with a clear verb and noun (e.g., "Login" or "Add Item").
- Draw these functions as horizontal ovals inside the system boundary.

Draw Relationships and Associations

- Connect actors to their specific use cases using simple solid lines.

- Add include or extend dashed arrows if certain use cases depend on or optionalize others.

- Use generalization arrows for specialized actors or use cases.



Hospital Management

Purpose: *Describe major services (functionality) provided by a hospital's reception.*

Summary*: This UML use case diagram example shows actor and use cases for a hospital's reception.* ***Hospital Reception*** *subsystem or module supports some of the many job duties of a hospital receptionist. Receptionist schedules patient's appointment and admission to the hospital, collects information from the patient by phone and/or upon patient's arrival to the hospital.*

For the patient that will stay in the hospital ("inpatient") she or he should have a bed allotted in a ward. Receptionists might also receive patient's payments, record them in a database and provide receipts, file insurance claims and medical reports.



Online Shopping -

Credit Cards Processing

UML Use Case Diagram Example

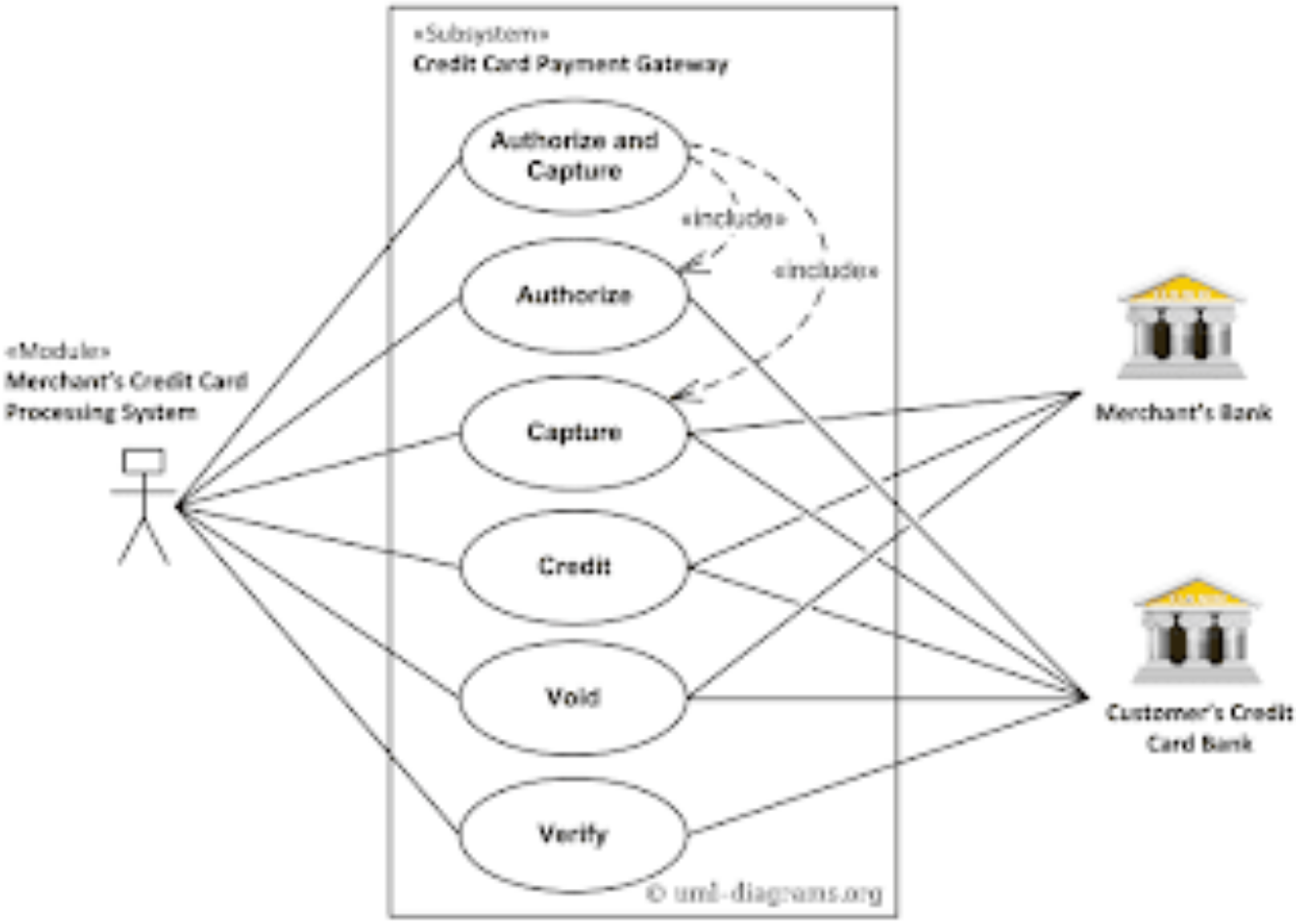
This UML use case diagram example shows some use cases for a system which processes credit cards.

Credit Card Processing System (aka Credit Card Payment Gateway) is a **subject**, i.e. system under design or consideration. Primary **actor** for the system is a **Merchant's Credit Card Processing System**. The merchant submits some credit card transaction request to the credit card payment gateway on behalf of a customer. Bank which issued customer's credit card is actor which could approve or reject the transaction. If transaction is approved, funds will be transferred to merchant's bank account.

Authorize and Capture use case is the most common type of credit card transaction. The requested amount of money should be first authorized by **Customer's Credit Card Bank**, and if approved, is further submitted for settlement. During the settlement funds approved for the credit card transaction are deposited into the **Merchant's Bank** account.

In some cases, only **authorization** is requested and the transaction will not be sent for settlement. In this case, usually if no further action is taken within some number of days, the authorization expires. Merchants can submit this request if they want to verify the availability of funds on the customer's credit card, if item is not currently in stock, or if merchant wants to review orders before shipping.

Capture (request to capture funds that were previously authorized) use case describes several scenarios when merchant needs to complete some previously authorized transaction - either submitted through the payment gateway or requested without using the system, e.g. using voice authorization.



UML use case diagram example for a credit cards processing system.

Credit use case describes situations when customer should receive a refund for a transaction that was either successfully processed and settled through the system or for some transaction that was not originally submitted through the payment gateway.

Void use case describes cases when it is needed to cancel one or several related transactions that were not yet settled. If possible, the transactions will not be sent for settlement. If the Void transaction fails, the original transaction is likely already settled.

Verify use case describes zero or small amount verification transactions which could also include verification of some client's data such as address.

You can find excellent resources, documentation, white papers, guides, etc. related to the credit card processing at **Authorize.Net - Payment Gateway to Accept Online Payments.**

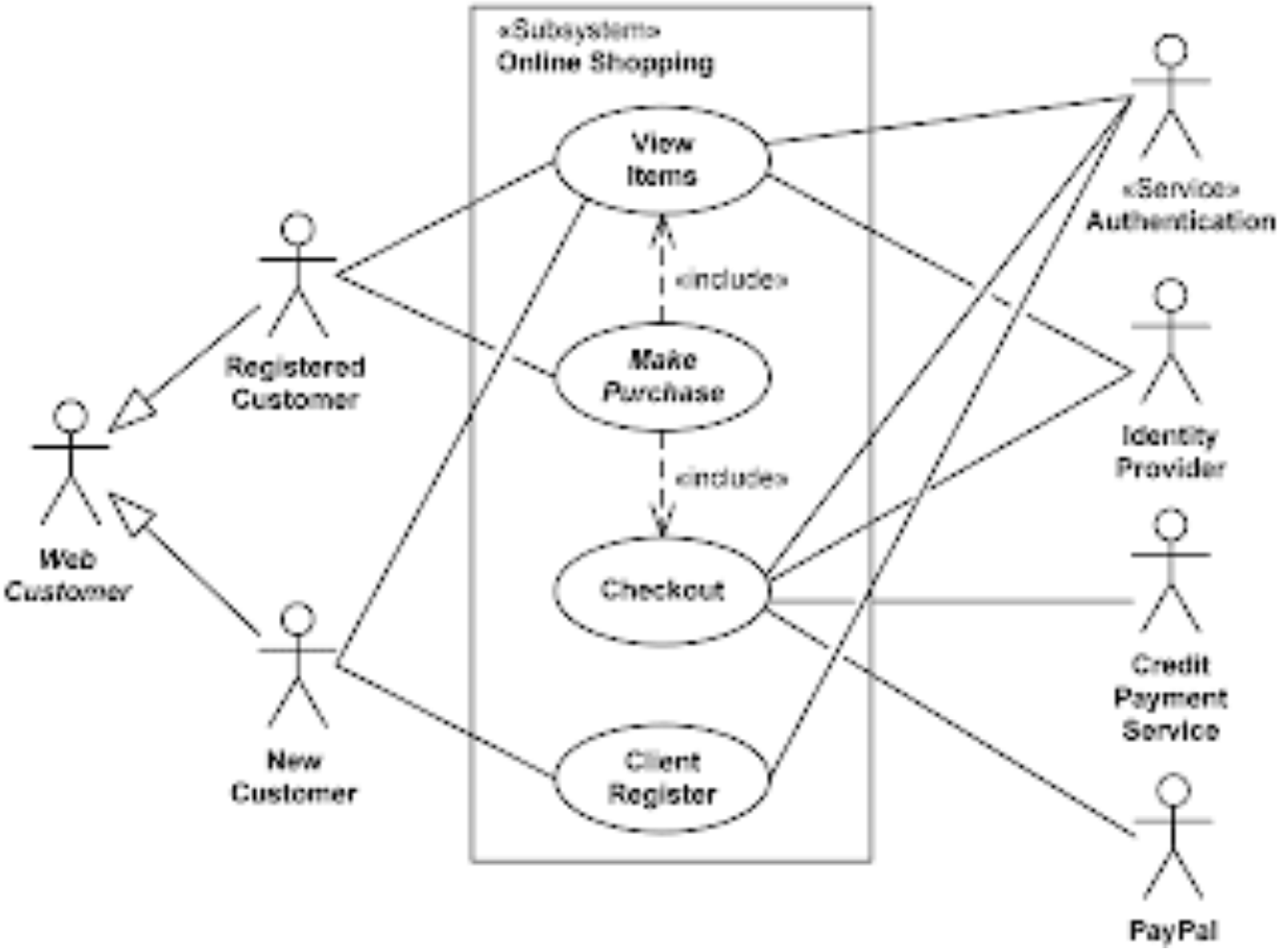
Online Shopping

UML Use Case Diagram Example

Web Customer **actor** uses some web site to make purchases online. Top level **use cases** are **View Items**, **Make Purchase** and **Client Register**. View Items use case could be used by customer as top level use case if customer only wants to find and see some products. This use case could also be used as a part of Make Purchase use case. Client Register use case allows

customer to register on the web site, for example to get some coupons or be invited to private sales. Note, that **Checkout** use case is **included use case** not available by itself - checkout is part of making purchase.

Except for the **Web Customer** actor there are several other actors which will be described below with detailed use cases.



Online shopping UML use case diagram example - top level use cases.

View Items use case is **extended** by several optional use cases - customer may search for items, browse catalog, view items recommended for him/her, add items to shopping cart or wish list. All these use cases are extending use cases because they provide some optional functions allowing customer to find item.

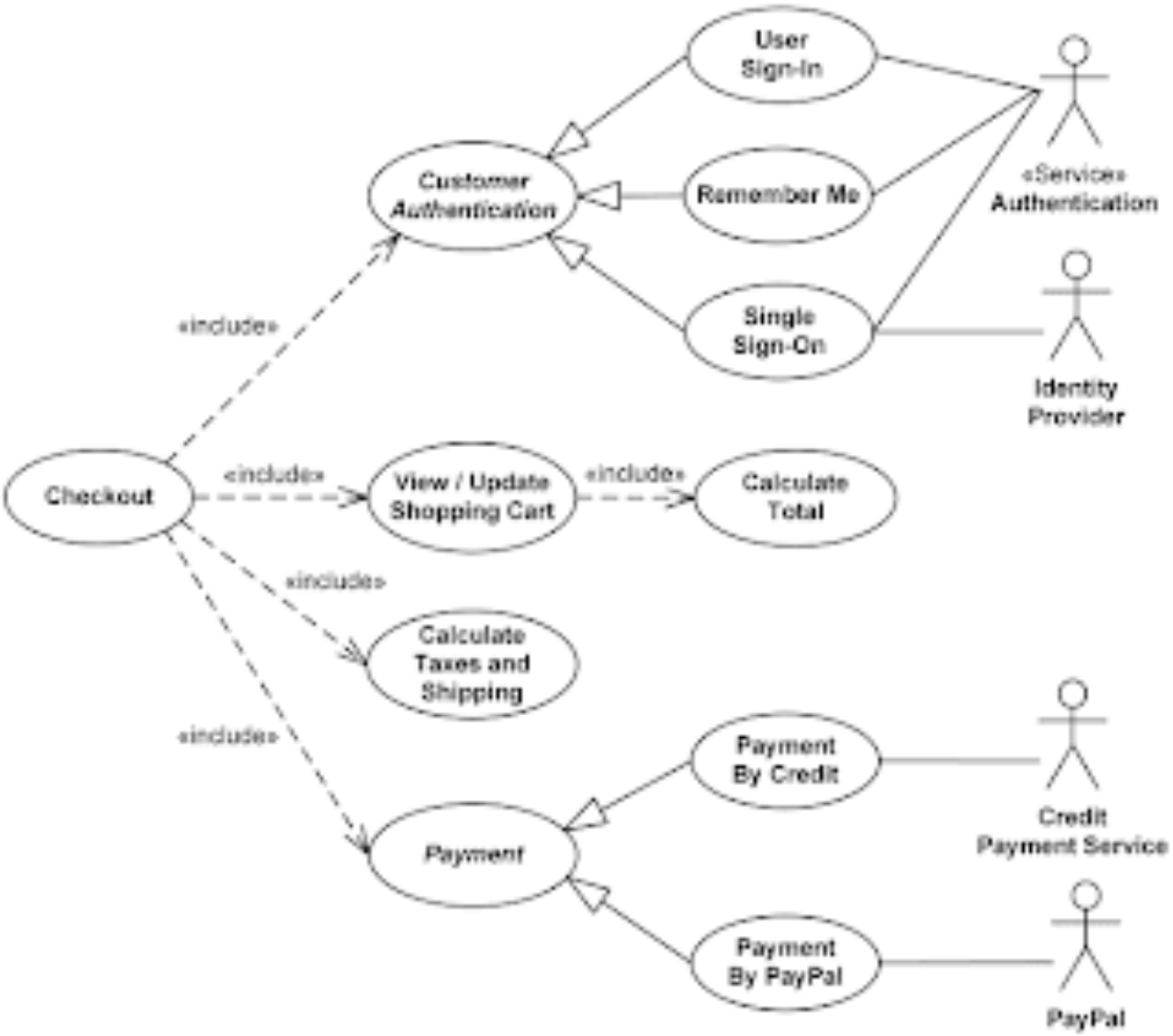
Customer Authentication use case is **included** in **View Recommended Items** and **Add to Wish List** because both require the customer to be authenticated. At the same time, item could be added to the shopping cart without user authentication.



Online shopping UML use case diagram example - view items use case.

Checkout use case includes several required uses cases. Web customer should be authenticated. It could be done through user login page, user authentication cookie ("Remember me") or Single Sign-On (SSO). Web site authentication service is used in all these use cases, while SSO also requires participation of external identity provider.

Checkout use case also includes **Payment** use case which could be done either by using credit card and external credit payment service or with PayPal.



An Activity Diagram is a behavioral UML diagram used to represent workflows, processes, and the sequence of activities within a system. It shows how actions are connected and how control flows from one activity to another.

- Illustrates sequential and concurrent activities, along with decision paths and control flow.
- Starts with an initial node and ends with a final node, helping model system behavior over time.

***Example**: An Online Shopping System activity diagram showing steps such as Browse Products-> Add to Cart -> Make Payment -> Order Confirmation.*

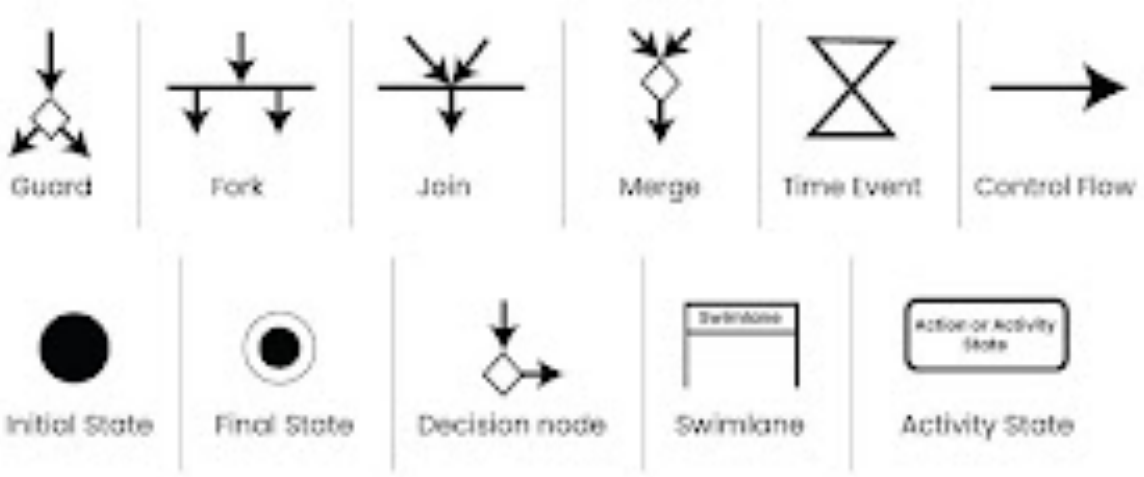
Uses of Activity Diagram

Activity diagrams are used to visualize workflows, processes, and the dynamic

behavior of a system.

- **Modeling Workflows or Processes:** Represents the sequence of activities in a business process or system workflow.
- **Concurrent or Parallel Processing:** Shows multiple activities that can execute simultaneously.
- **Understanding Dynamic Behavior:** Illustrates how a system behaves and responds to events over time.
- **Clarifying Complex Logic:** Simplifies decision-making and branching processes for better understanding.
- **System Design and Analysis:** Helps analyze, design, and communicate system behavior effectively.
- **Describing Use Cases:** Visualizes the flow of actions and interactions within a use case.

Activity Diagram Notations

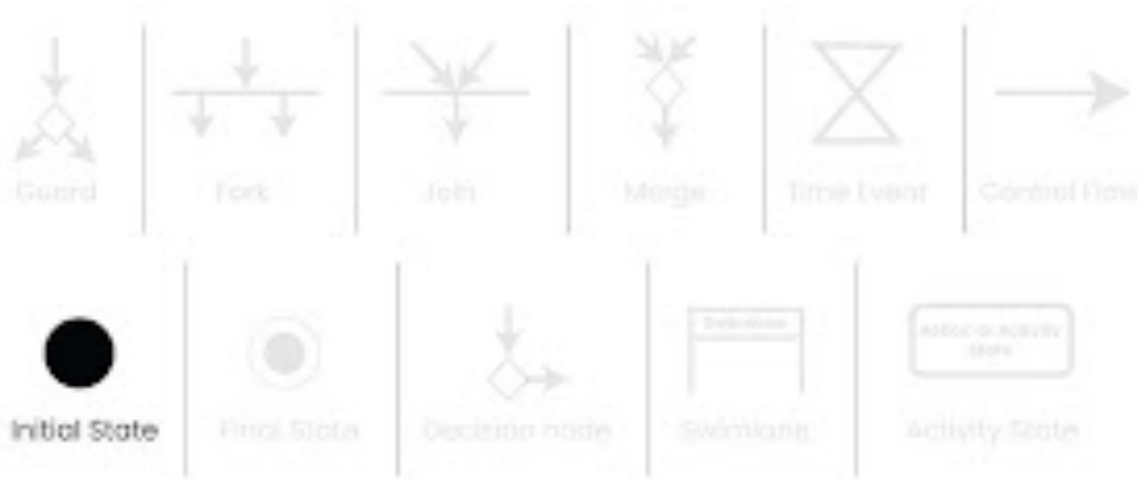


Activity Diagram Notations

1. Initial State

Represents the starting point of an activity or process.

- Shown as a filled black circle in the diagram.
- A process typically has only one initial state unless nested activities are used.



Initial State

Example: *Here the initial state of the system before the application is opened.*

Initial State symbol being used



Unified Modeling Language (UML) | Activity Diagrams

2. Action or Activity State

Represents the execution of an action or task within a process.

- Shown as a rectangle with rounded corners.
- Represents any action, operation, or event that occurs in the workflow.



Activity State

***Example**: Consider the previous example of opening an application, opening the application is an activity state in the activity diagram.*

Activity State symbol being used



Unified Modeling Language (UML) | Activity Diagrams

3. Action Flow or Control flows

Represents the transition and sequence of activities within a process.

- Shown as a line with an arrowhead indicating the direction of flow.
- An activity can have multiple incoming and outgoing control flows, with conditions specified on the arrows if required.



Control Flow

***Example**: Here both the states transit into one final state using action flow symbols i.e. arrows.*

Using Action Flows for Transitions

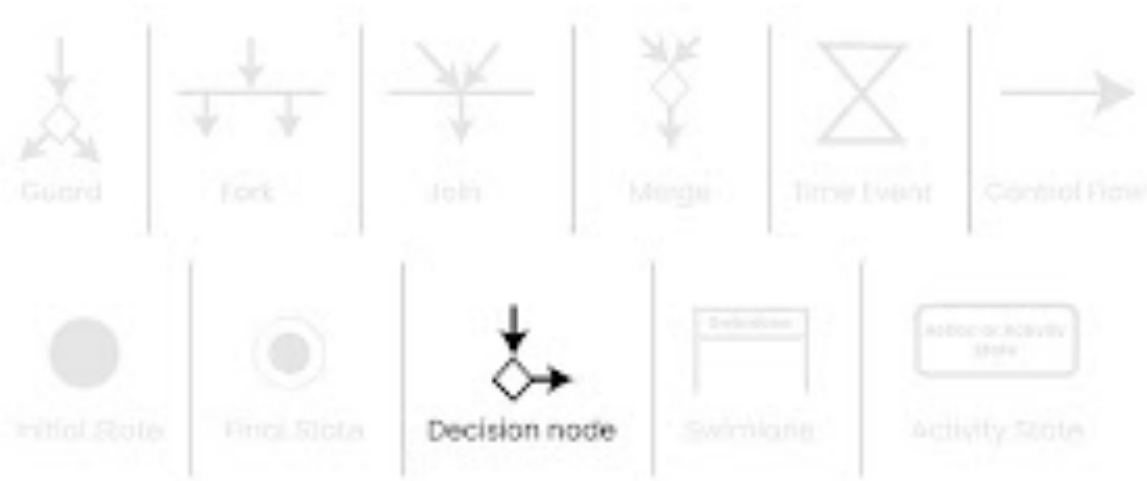


Unified Modeling Language (UML) | Activity Diagrams

4. Decision node and Branching

Represents a point where a decision is made to determine the next flow of control.

- Shown with multiple outgoing paths based on conditions or guard expressions.
- Always has two or more outgoing control flow arrows.

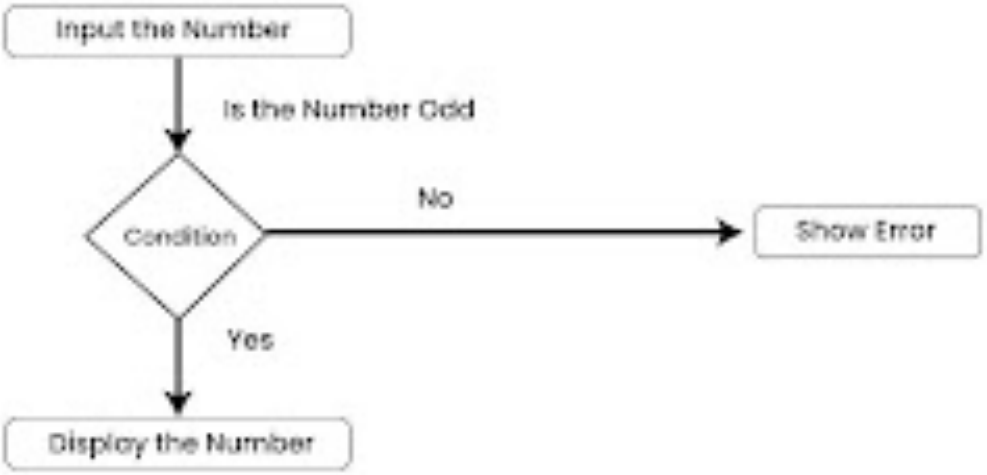


Decision Node

***Example**: We apply the conditions on input number to display the result :*

- *If number is odd then display the number.*
- *If number if even then display the error.*

An Activity Diagram using Decision Node

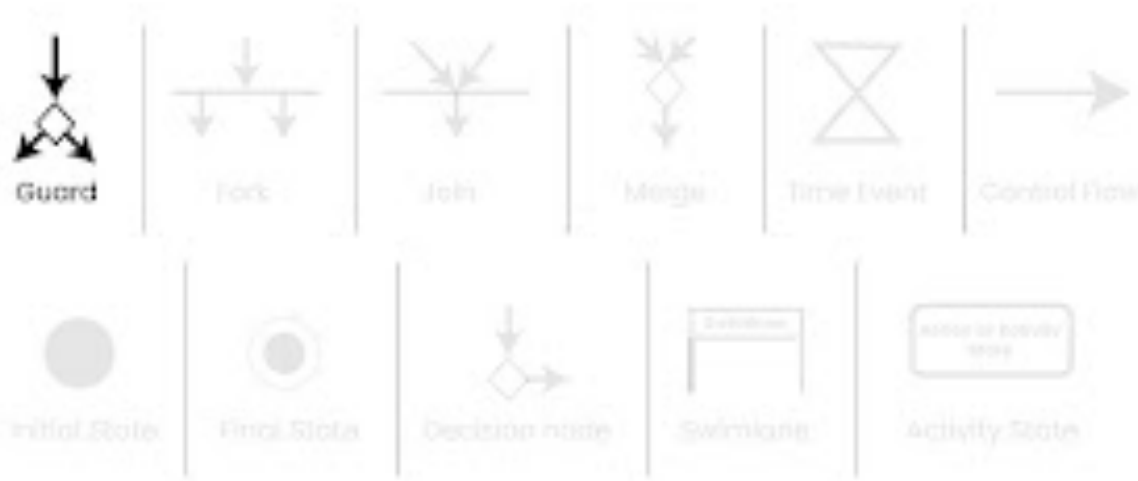


Unified Modeling Language (UML) | Activity Diagrams

5. Guard

Represents a condition that must be satisfied for a particular flow to be followed.

- Written next to a decision path, often within square brackets.
- Helps control the direction of flow based on specific conditions.



Guard

6. Fork

Represents a point where a workflow is split into multiple parallel activities.

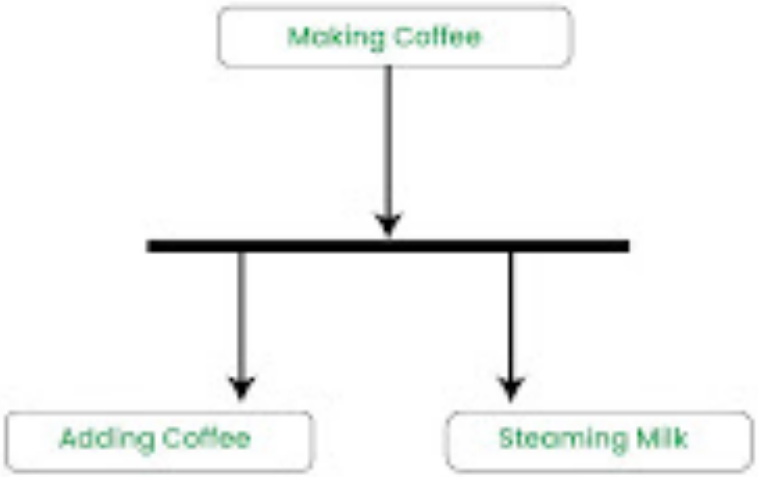
- Shown as a solid rectangular bar with one incoming arrow and multiple outgoing arrows.
- Allows several activities to execute simultaneously.



Fork

Example: *In the example below, the activity of making coffee can be split into two concurrent activities and hence we use the fork notation.*

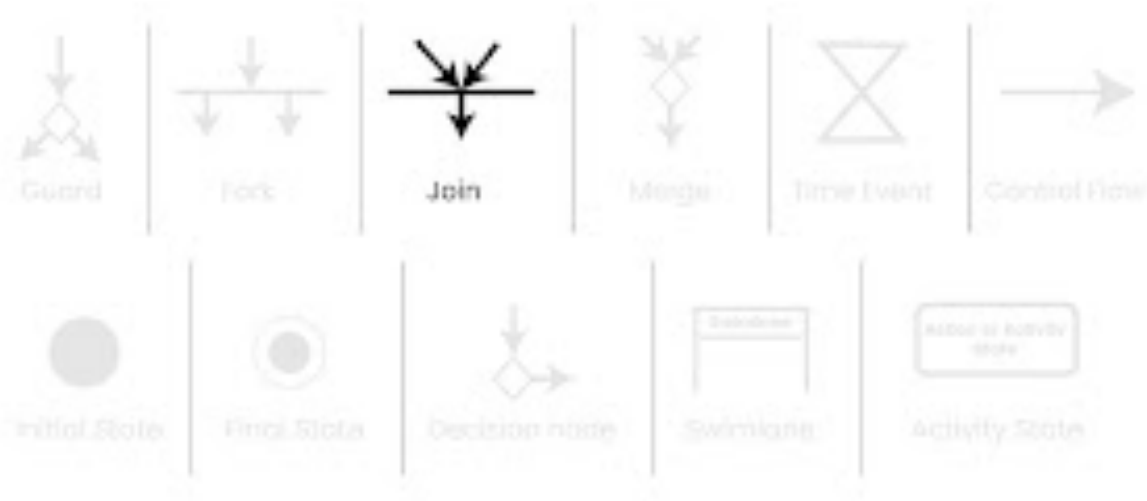
A Diagram using Fork Notation



7. Join

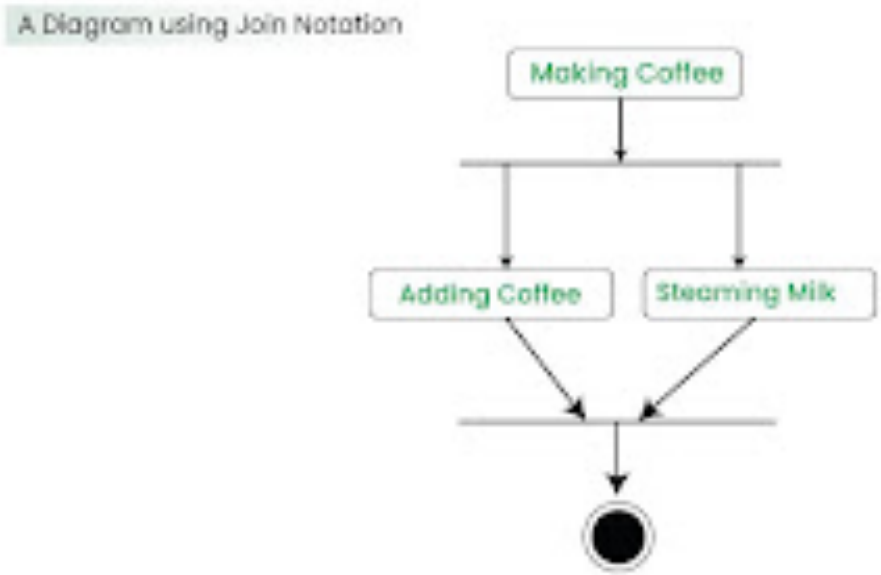
Represents a point where multiple parallel activities merge into a single flow.

- Shown with two or more incoming edges and one outgoing edge.
- Synchronizes concurrent activities before proceeding to the next step.



Join

Example: *When both activities i.e. steaming the milk and adding coffee get completed, we converge them into one final activity.*

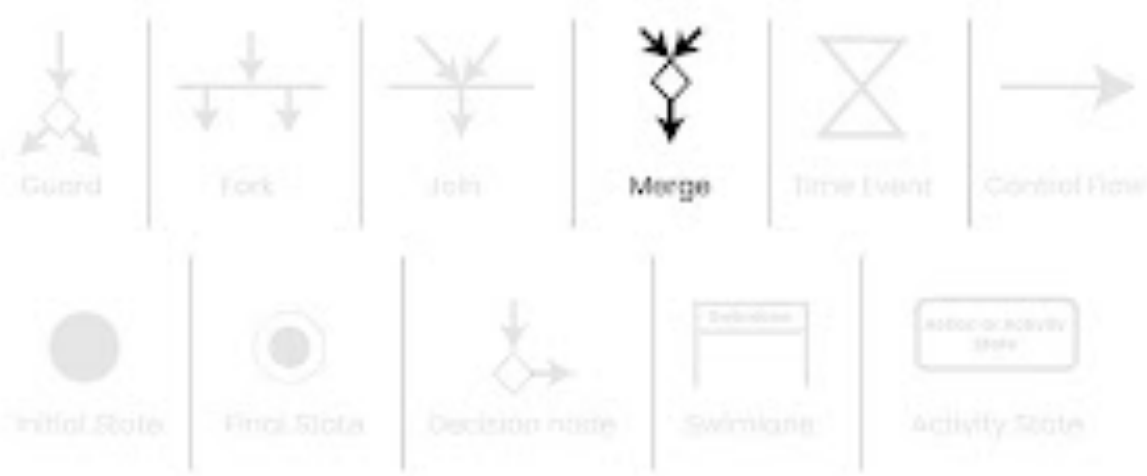


Unified Modeling Language (UML) | Activity Diagrams

8. Merge or Merge Event

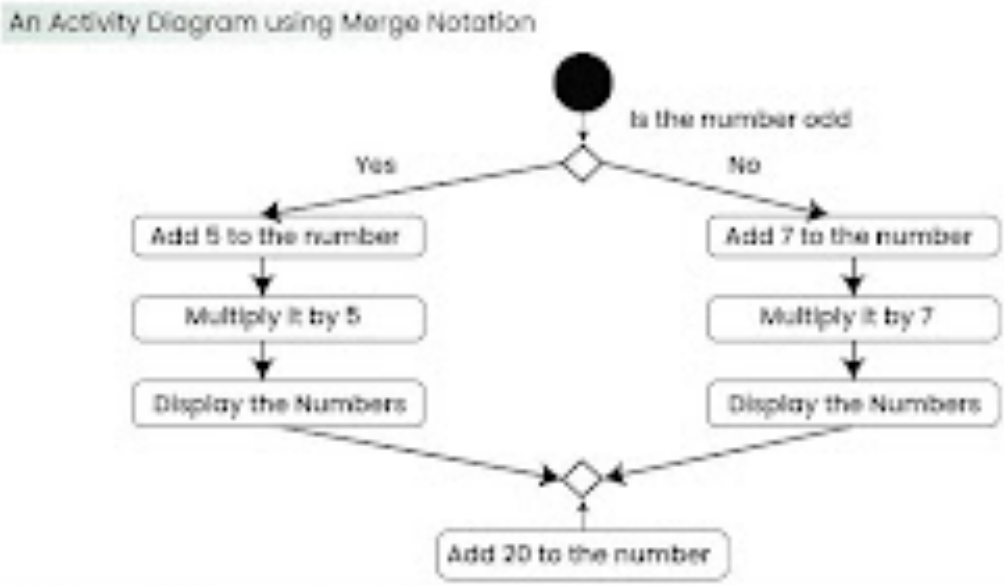
Represents a point where multiple alternative paths combine into a single flow.

- Merges two or more workflow paths into one outgoing path.
- The process continues regardless of which incoming path was followed.



Merge

Example: *In the diagram below: we can't have both sides executing concurrently, but they finally merge into one. A number can't be both odd and even at the same time.*

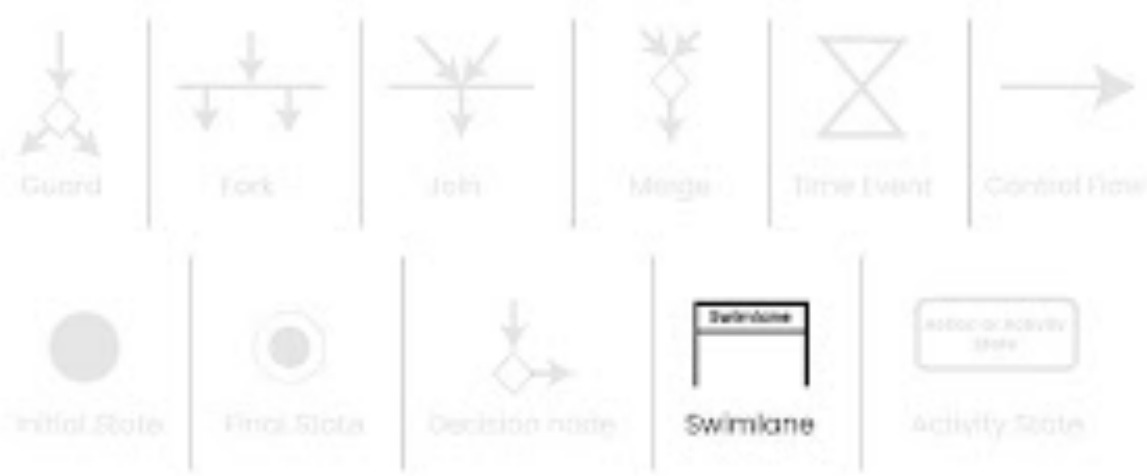


Unified Modeling Language (UML) | Activity Diagrams

9. Swimlanes

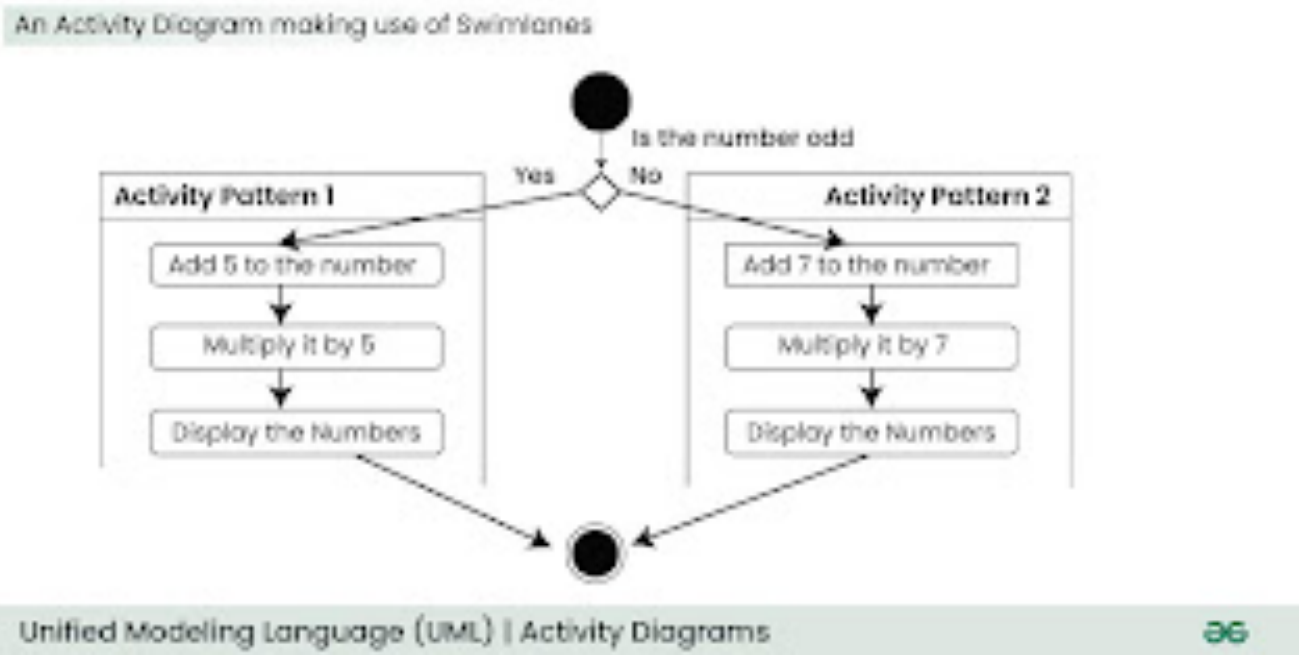
Represents a grouping of activities based on roles, responsibilities, or processes.

- Shown as a rectangular row or column in the activity diagram.
- Helps identify who is responsible for each activity and improves workflow clarity.



Swimlane

Example: *Here different set of activities are executed based on if the number is odd or even. These activities are grouped into a swimlane.*



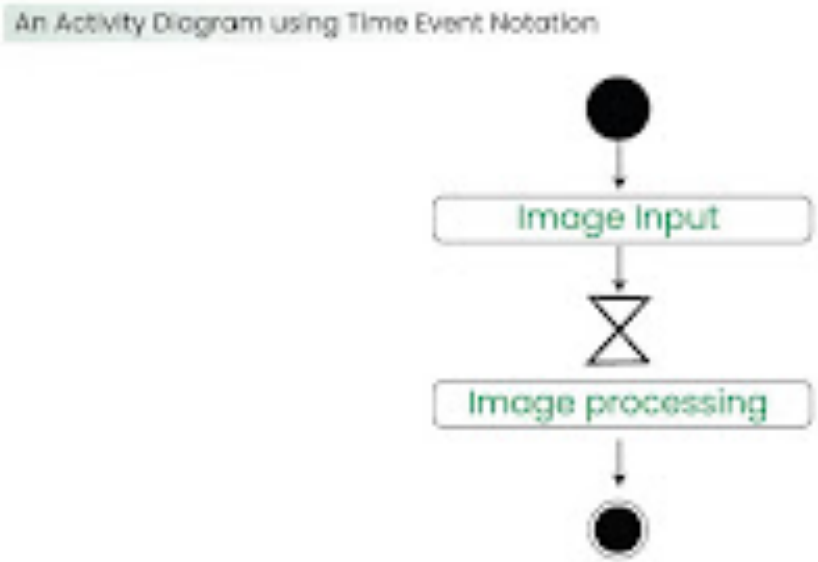
10. Time Event

Represents an event that pauses the workflow for a specific period of time.

- Shown using an hourglass symbol in the activity diagram.
- Used when an activity or event requires a time delay before continuing.



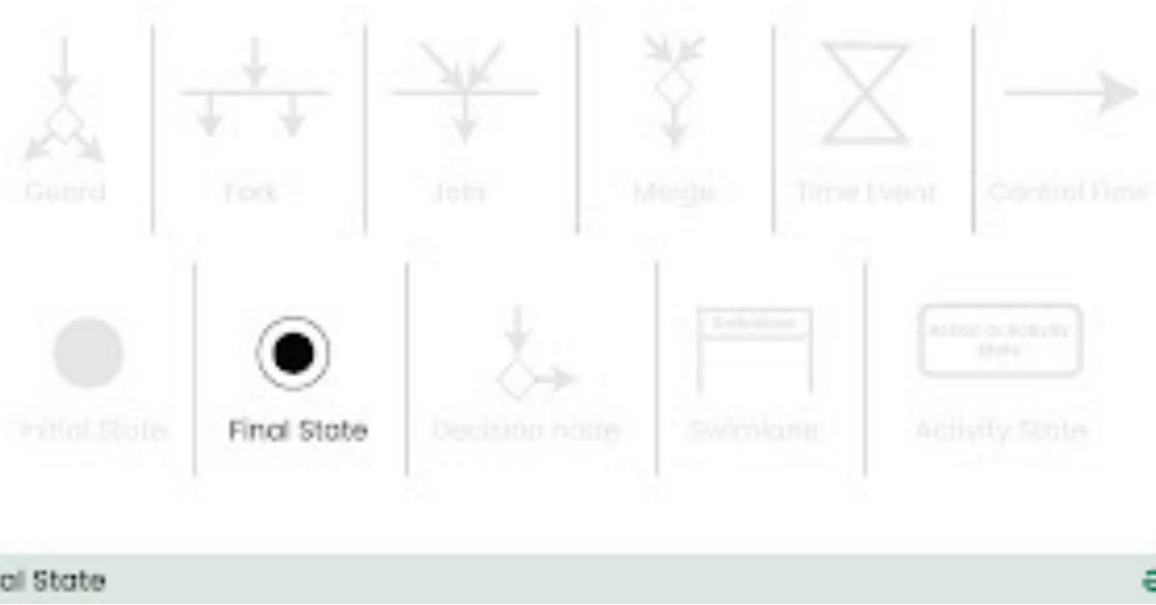
Example: *Let us assume that the processing of an image takes a lot of time. Then it can be represented as shown below.*



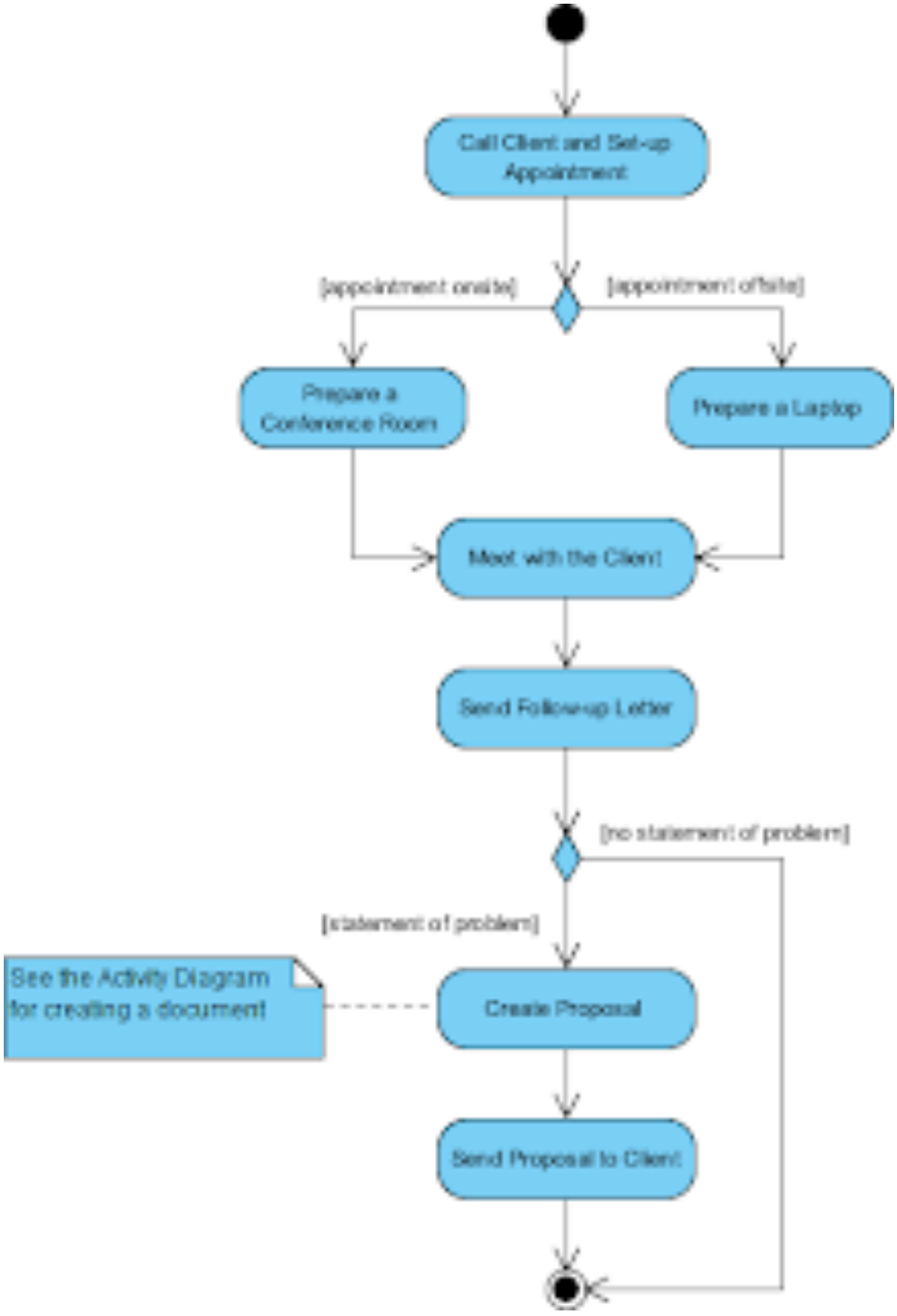
11. Final State or End State

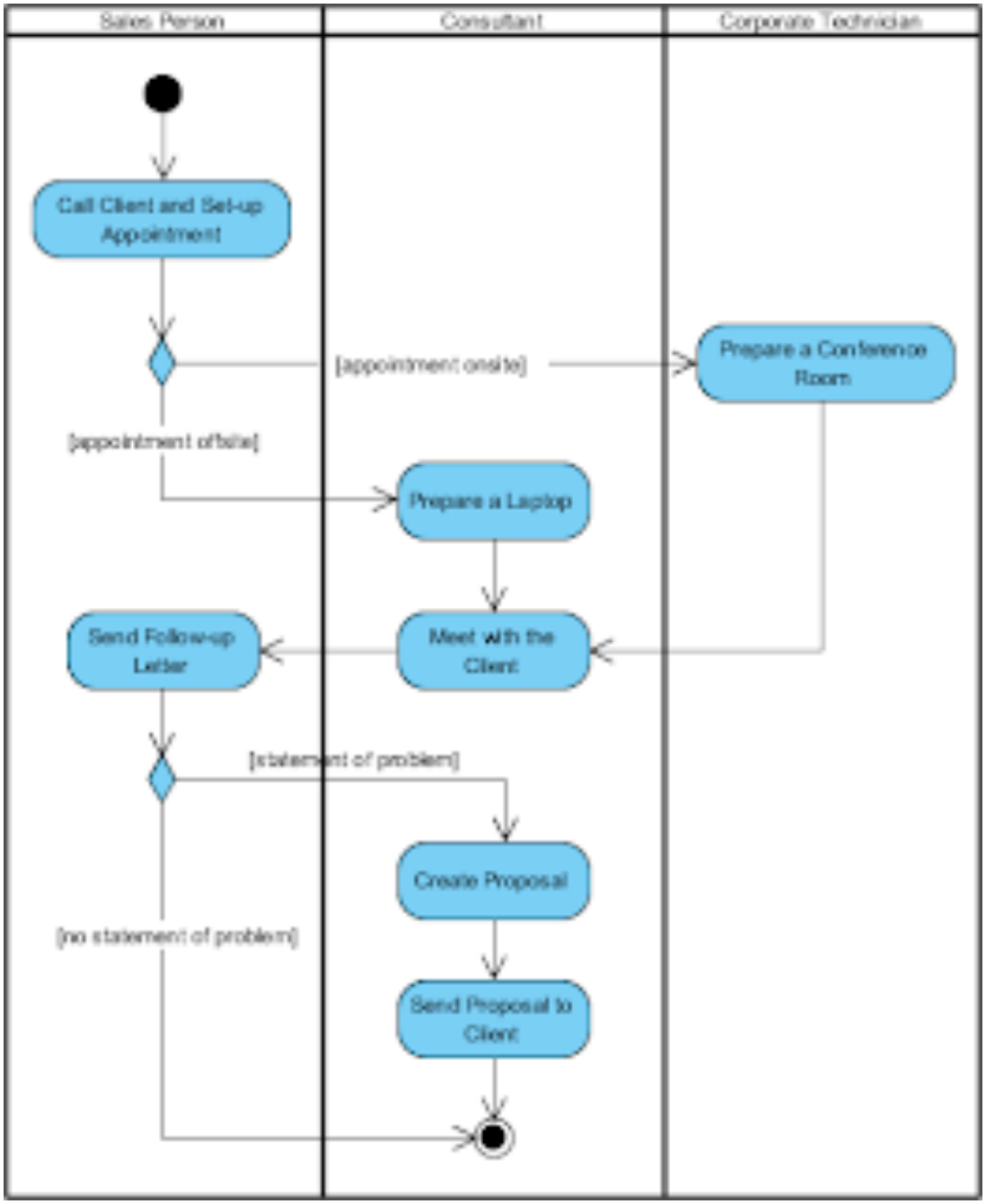
Represents the point where a process or activity reaches completion.

- Shown as a filled circle inside another circle.
- A system or process can have multiple final states.



Swimlane and Non-Swimlane Activity Diagram


Call Client and Set-up Appointment
[appointment onsite]
[appointment offsite]
Prepare a Conference Room
Prepare a Laptop
Meet with the Client
Send Follow-up Letter
[no statement of problem]
[statement of problem]
See the Activity Diagram for creating a document
Create Proposal
Send Proposal to Client



Differences between an Activity diagram and a Flowchart

An activity diagram is very similar to a flowchart. So let us understand if activity diagrams or flowcharts are any different.

Flow Chart

An algorithm is like a set of clear instructions to solve a problem, and a flowchart is a picture that shows those instructions.

- Flowcharts help programmers visualize the steps of an algorithm and plan the logic required to solve a problem before coding.
- They are also used to model business workflows and decision-making processes, making them a simple precursor to activity diagrams.

***Example**: A manufacturer uses a flow chart to explain and illustrate how a particular product is manufactured.*

Flowchart	Activity Diagram
Represents the step-by-step flow of a process or algorithm	Represents the flow of activities and actions in a system
Used mainly for procedural or algorithmic logic	Used to model workflows and business processes
Focuses on control flow using symbols like start, process, decision	Focuses on activities, decisions, parallel flows, and transitions
Simple and easy to understand	More expressive and detailed than flowcharts
Commonly used in programming and problem-solving	Commonly used in UML for system and process modeling
Does not support concurrency well	Supports parallel and concurrent activities

Class Based Modelling / Element

- Class based modeling represent classes, object, attributes & operations of system.

1. Class Diagram:

- The class diagram shows a static view of an application.
- It represents the types of objects residing in the system and the relationships between them.
- It describes the major responsibilities of a system.



